



PostDoc Report

G.Botti

July 18, 2026

Contents

1	CyberSchool 2026 Project	3
1.1	Project outline	3
1.2	<code>runner.py</code> : a heavily customizable dynamics script	3
1.3	Quasiclassical trajectories	5
1.4	The H_2 in $H_2 \cdot C_{60}$ spectra	7
1.5	Running OpenMolcas	9
1.6	The toy Hamiltonian	13
1.7	The toy Spectral density	13
1.8	Exploratory TENS0 simulation	15
1.9	Conclusions	17
1.10	Contents of the folder	17

List of Figures

1.1	CyberTraining 2026: H_2 spectra	4
1.2	CyberTraining 2026: H_2O spectra	5
1.3	CyberTraining 2026: optimized geometry	6
1.4	CyberTraining 2026: VDOS	7
1.5	CyberTraining 2026: H_2 influence spectrum	8
1.6	CyberTraining 2026: H_2 stretching	8
1.7	CyberTraining 2026: H_2 PES	12
1.8	CyberTraining 2026: H_2 gap	12
1.9	CyberTraining 2026: BO components vs VDOS	14
1.10	CyberTraining 2026: spectral density components	15
1.11	CyberTraining 2026: TENS0 outputs	17

List of Tables

1.1	CyberTraining2026: OpenMolcas output summary	13
1.2	CyberTraining 2026: Spectral density parameters	14

CHAPTER 1

CYBERSCHOOL 2026 PROJECT

1.1 PROJECT OUTLINE

My work is split in two. The first part is creating a quasiclassical trajectory (QCT) integrator using PySCF, and test it to obtain the quasiclassical *ab initio* spectrum of H_2 . The second part will consist in computing the open quantum system dynamics of $\text{H}_2 \cdot \text{C}_{60}$, to check if it is possible to compute the spectral density from an *ab initio* trajectory. I will compute the S_0 and S_1 harmonic frequencies for isolated H_2 using OPENMOLCAS (this is an approximation, but it will save me time). I will then move to compute the spectral density of $\text{H}_2 \cdot \text{C}_{60}$ by Fourier-transforming the QM energy fluctuations in an ONIOM QCT dynamics. The theoretical background is described in [this tutorial](#) (hoping that the fluctuations of the QM energy are suitable). The ONIOM simulation will be carried out with ORCA and [Dragonball](#), so that it can run in parallel to the other parts of the project. In case the spectral density could not be computed as expected, the vibrational density of states (VDOS) of C_{60} would be used instead (this should not be a great approximation, since C_{60} VDOS is already pretty simple). As for the system-bath coupling, some tuning would be necessary, if time allows and the results are interesting. With the H_2 harmonic frequencies and some spectral density peaks, I can build the model Hamiltonian in TENSOR and run a small simulation.

1.2 RUNNER.PY: A HEAVILY CUSTOMIZABLE DYNAMICS SCRIPT

The quasiclassical trajectory method consists on running a Born-Oppenheimer classical dynamics starting from initial quantized conditions, usually:

$$\mathbf{q}_0 = \mathbf{q}_{\text{eq}} \qquad \mathbf{p}_0 = \sqrt{\omega}$$

where ω is the harmonic frequencies vector. Using this initial conditions is not easy at all, because standard *ab initio* packets do not take initial velocities as input (except for QCHEM). This problem is already solved by [Dragonball](#), but more control over the dynamics, its input, and its output is always desirable. For this reason, the first task of this project is writing a PYTHON code that uses PySCF to compute the energy and the forces required to evolve the dynamics with a Velocity Verlet algorithm. The code (aptly named `runner.py`) takes in input the initial positions (`geometry.xyz`)

and velocities (`velocity.xyz`) in `.xyz` format, making it fully compatible with `Dragonball` internal outputs. It outputs only the trajectory (containing positions, velocities and energies, again in `.xyz` format), the forces, and the energy log. At the moment, it is only capable of reading the initial velocities, but I plan to expand it to include the optimization and frequency task required to initialize the trajectory. In Figure 1.1, I compare the H_2 spectra obtained with `runner.py` and ORCA, both at DFT-D4/B3LYP def2-TZVP level of theory. To showcase the code capabilities, Figure 1.2 presents the quasiclassical and semiclassical (based on Heller frozen Gaussian) spectra of H_2O , obtained with `runner.py` at DFT-D4/B3LYP def2-TZVP level of theory. In the future, I plan to use the `PySCF.mol` object to easily run the dynamics with other *ab initio* packages. Moreover, the increased control over the dynamics evolution would allow me to update my FORTRAN code to perform on-the-fly Adiabatic Switching, and even to easily run dynamics on excited electronic states. A refined version of `runner.py` will be included in `Dragonball`.

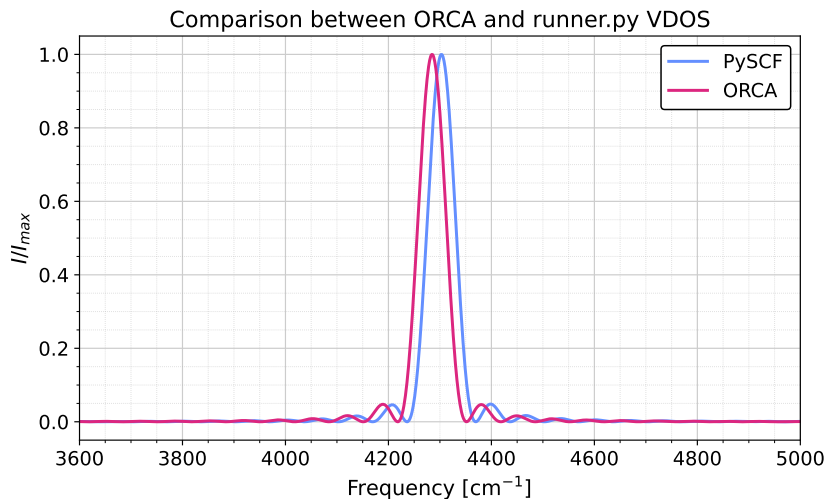


FIGURE 1.1: Quasiclassical trajectory vibrational densities of states of H_2 , obtained with `runner.py` and ORCA, at DFT-D4/B3LYP def2-TZVP level of theory

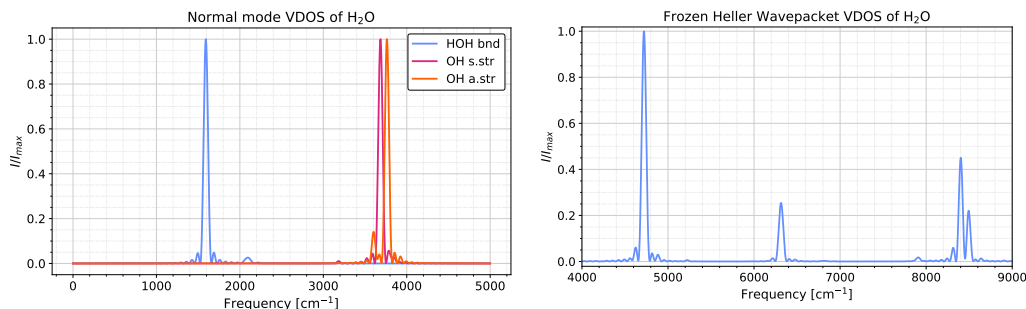


FIGURE 1.2: Left: quasiclassical trajectory VDOS of H_2O , computed on a `runner.py` trajectory at DFT-D4/B3LYP def2-TZVP level of theory, and projected into normal modes; right: Heller's frozen wavepacket VDOS computed on the same trajectory (notice the prominent zero-point energy peak)

1.3 QUASICLASSICAL TRAJECTORIES

For the quasiclassical trajectory, I use the standard `Dragonball` procedure with ORCA. The `runner.py` script is set up to seamlessly take the place of the *ab initio* package, at least for more mundane simulations setups, but in this case ORCA was chosen to work in parallel and save time. An initial guess for $\text{H}_2 \cdot \text{C}_{60}$ is generated with AVOGADRO and optimized with xTB using

```
1 xtb --gfn 2 input.xyz -P N --iterations 1500 --opt tight --charge 0 --molden >
  ↳ xtbopt.xyz
```

where N is the number of cores. Starting from this geometry, a conformational scan is performed using CREST

```
1 crest xtbopt.xyz --gfn 2 -T N --iterations 1500 >& output&
```

The conformational scan is useless in this case, but it is a good practice nonetheless. The optimization and frequencies input for ORCA are generated using `Bulma`:

```
1 python bulma.py H2_C60_xtb_crest.xyz --orca-freq
```

Since `Bulma` still does not support either the xTB or ONIOM inputs, I manually modify the `.inp` files with

```
1 ! VERYTIGHTOPT FREQ QM/XTB B3LYP D4 Def2-TZVPD VeryTightSCF
2 %QMMM QMATOMS {60 61} END END
```

The equilibrium geometry (Figure 1.3) and Hessian matrix are extracted and prepared for the next step using again `Bulma`:

```
1 python bulma.py geom_freq.out --extract-geo
2 python bulma.py geom_freq.hess --orca-hess
```

In the case of the ONIOM optimization, `Bulma` extracts only the QM geometry, so the full optimized geometry must be inserted manually in the `.inp` files. The quantized initial velocities are computed using `Vegeta`. As usual, the modes with harmonic frequencies below 400 cm^{-1} are initialized with zero initial velocity. This is done to prevent the interference of the librational modes on the final

VDOS. This practice originates in the Multiple Coherent Semiclassical Initial Value Representation procedure, but became a good practice in the quasiclassical trajectory one too.

```
1 python vegeta.py --xyz geo_opt.xyz --hess Hessian_flat.out --freq-thresh 400
```

Then, Bulma is used to parse the trajectory output and prepare it for Flying ν_i mbus:

```
1 python bulma.py dummy --parse-orca-qmd
```

The same procedure is performed for H_2 (level of theory: DFT-D/B3LYP def2-TZVP) and C_{60} (level of theory: xTB). *In this last case, the keywords*

```
1 ! VERYTIGHTOPT FREQ GFN2-xTB VeryTightSCF
```

are manually inserted. The atom-wise spectra computed with Flying ν_i mbus are shown in Figure 1.4.

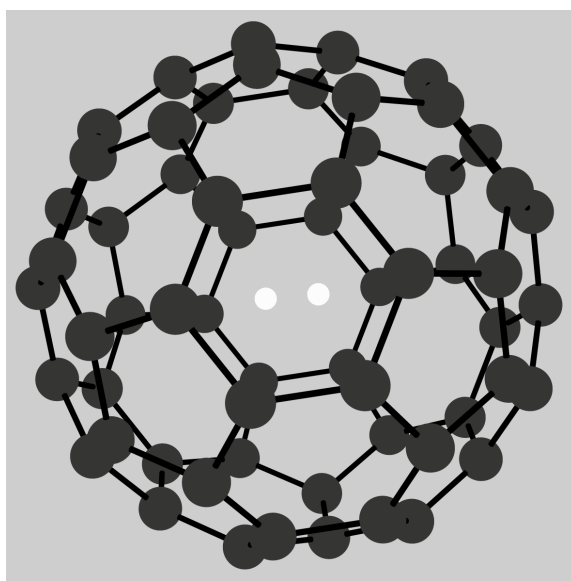


FIGURE 1.3: *Optimized Geometry for $H_2 \cdot C_{60}$*

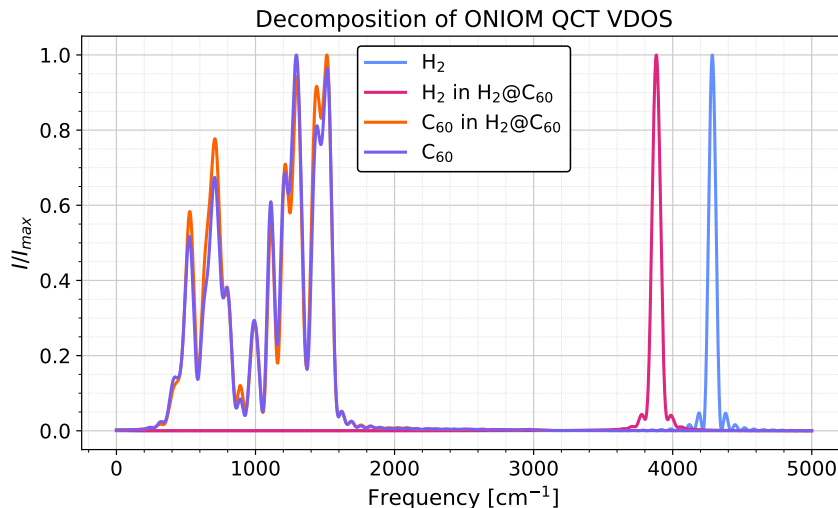


FIGURE 1.4: Quasiclassical Trajectory VDOS for H_2 , C_{60} , H_2 in $H_2 \cdot C_{60}$ and C_{60} in $H_2 \cdot C_{60}$

1.4 THE H_2 IN $H_2 \cdot C_{60}$ SPECTRA

Inspired by this CyberTraining 2026 [tutorial](#), I try to compute the spectral density of C_{60} from the fluctuation of the quantum (QM1) energy along the ONIOM trajectory. ORCA prints these values inside the `.scf.log` file, so I start by creating a PYTHON script (`prepare_fft.py`) to extract them, subtract the average from them and prepare the time series for the FFT. Unfortunately, the resulting influence spectrum does not show any peak in the region of C_{60} vibrational frequency (Figure 1.5). If this were an actual spectral density, it would mean that there is no vibrational coupling between H_2 and C_{60} . Given the unorthodox way in which I obtained this spectrum, I suspend the judgment and try to confirm the lack of vibrational coupling by looking at the VDOS of the H_2 stretching of $H_2 \cdot C_{60}$ (Figure 1.6). The lack of any C_{60} signal in the stretching VDOS hints that the normal mode is completely decoupled along the trajectory, that leads me to believe that the redshift of H_2 stretching is mostly due to long range interactions, or some unexpected effect in the ONIOM energy. This lack of coupling is consistent with the relative position of H_2 inside the C_{60} cage (Figure 1.3): H_2 is more or less isolated at the center. Nonetheless, the unusual redshift will be investigated in the future. The hunt for a sound method on how to extract the vibrational coupling (and spectral density) from an *ab initio* trajectory continues.

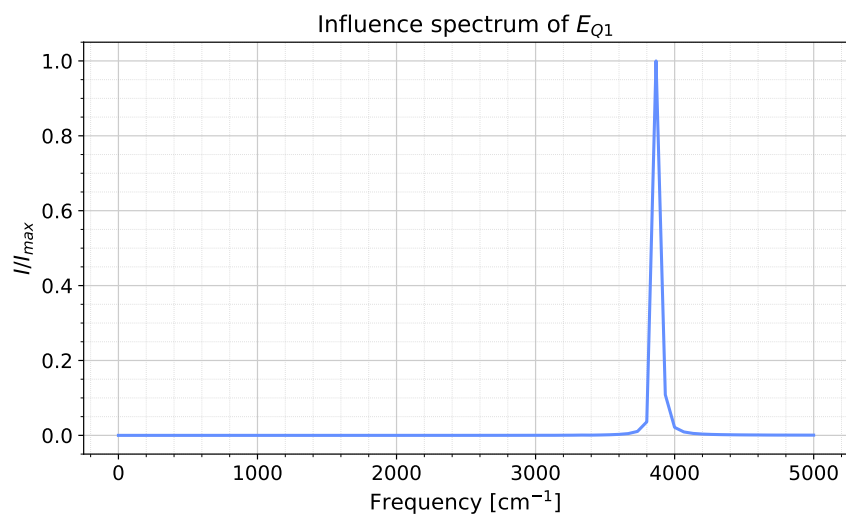


FIGURE 1.5: *Fourier transform of the QM1 (DFT-D4/B3LYP def2-TZVP) energy fluctuations for H_2 caged in C_{60}*

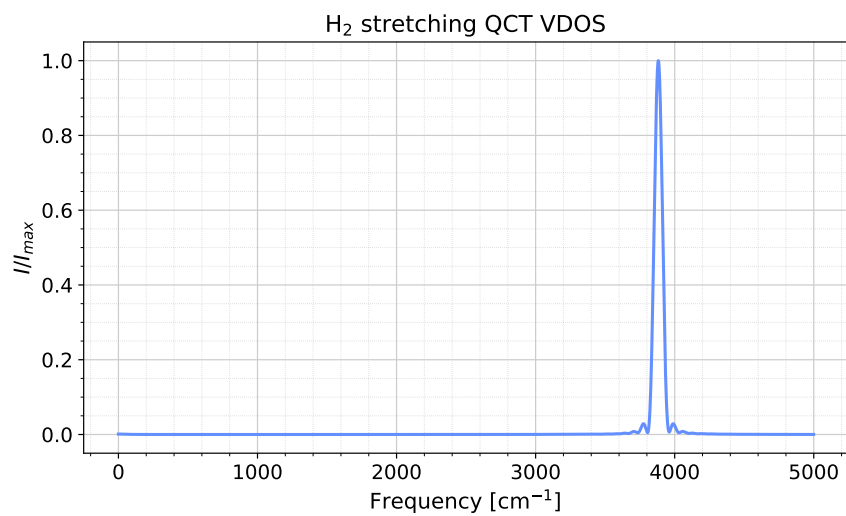


FIGURE 1.6: *Fourier transform the H_2 stretching momentum for $H_2 \cdot C_{60}$*

1.5 RUNNING OPENMOLCAS

To build the toy Hamiltonian for my system, I need the harmonic frequencies of the S_0 and S_1 electronic levels for H_2 , and something to represent the diabatic coupling. Since I don't want to mess with ONIOM in CASSCF, I use the isolated H_2 . The approximations are piling up!

I run OPENMOLCAS on CINECA's Galileo100 supercomputer. To do so, I use

```
1 module load profile/chem-phys
2 module load autoload openmolcas/21.10
3 module help openmolcas
```

The harmonic frequencies for S_0 are computed using the following input:

```
1 &GATEWAY
2 Coord
3 2
4 Hydrogen
5 H -0.3900956909 0.0026329421 0.0011992262
6 H 0.3900911392 -0.0026058577 -0.0012204178
7 Basis = def2-TZVP
8 Group = NoSym
9
10 >>> Do While
11 &SEWARD
12 >>> IF ( ITER = 1 ) <<<
13 &SCF; KSDFT = B3LYP; Charge = 0; Spin = 1
14 >>> ENDIF <<<
15 &RASSCF
16 Title = H2 SA-3-CAS(2,2)SCF
17 symmetry = 1
18 Nactel = 2 0 0
19 Spin = 1
20 Inactive = 0
21 Ras2 = 4
22 ciroot = 2 2 1
23 RLXRoot = 1
24 &SLAPAF
25 >>> EndDo
26
27 &MCKINLEY
28 ShowHessian
```

I use `Inactive = 0` because there are no inactive orbitals in H_2 . Since I am using a very large basis set (`Basis = def2-TZVP`), I set `Ras2 = 4`. I follow the gradient of the first root (`RLXRoot = 1`) in the RASSCF job with two roots. Following the OpenMolcas hands-on tutorial, I use CASPT2 to compute the harmonic frequency of S_1 , using the following input

```
1 &GATEWAY
2 Coord
3 2
4 Hydrogen
5 H -0.3900956909 0.0026329421 0.0011992262
6 H 0.3900911392 -0.0026058577 -0.0012204178
7 Basis = def2-TZVP
8 Group = NoSym
9
```

```

10 >>> Do While
11 &SEWARD
12 >>> IF ( ITER = 1 ) <<<
13 &SCF; KSDFT = B3LYP; Charge = 0; Spin = 1
14 >>> ENDIF <<<
15 &RASSCF
16 Title = H2 SA-2-CAS(2,2)SCF
17 symmetry = 1
18 Nactel = 2 0 0
19 Spin = 1
20 Inactive = 0
21 Ras2 = 4
22 ciroot = 2 2 1
23 RLXRoot = 1
24 &CASPT2
25 XMultiState = all
26 convergence = 1.0d-07
27 IPEA = 0.00
28 IMAGinary = 0.25
29 RLXRoot = 2
30 >> COPY $WorkDir/$Project.JobMix $WorkDir/JOB001
31 &ALASKA
32 CutOFF=1.0D-9
33 SHOW
34 &SLAPAF
35 >>> EndDo
36
37 &MCKINLEY
38 ShowHessian

```

I follow the gradient of the second root (RLXRoot =2) in the CASPT2 job with two roots.

Finally, I have to find something for the diabatic coupling. For a molecule such as H₂, the fastest way is to scan the two potential energy surfaces and compute the minimum gap average energy:

$$E_{12} \simeq \frac{E_1 - E_0}{2}$$

where E_1 and E_0 are the S_1 and S_0 energies at the point of minimum separation between the two surfaces. It is not the strictest approach, but I hope it will give me a sensible number to work with. With this scan, I can also compute the separation between the two potential energy surfaces, ΔE . I will compute it at the S_0 minimum.

Since I am not an OPENMOLCAS veteran, the quickest way I found to do a surface scan is to wrap my run into a bash script that updates the input file for each distance:

```

1 export MOLCAS_NPROCS=$SLURM_NTASKS
2
3 export CurrDir=$(pwd)
4
5 dists=(0.5 0.6 0.7 0.8 0.9 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2.0 2.1 2.2 2.3
6 ↪ 2.4 2.5)
7
8 printf "dist ES0 ES1 gap\n" > gaps.dat
9
10 for dist in "${dists[@]"; do
11   cat > h2.inp <<EOF

```

```

12      Coord
13      2
14      OpenCazzas
15      H 0.000000 0.000000 0.000000
16      H 0.000000 0.000000 $dist
17      Basis = def2-TZVP
18      Group = NoSym
19
20 &SEWARD
21
22 &SCF
23      KSDFT = B3LYP
24      Charge = 0
25      Spin = 1
26
27 &RASSCF
28      Title = H2 SA-2-CAS(2,2)SCF
29      symmetry = 1
30      Nactel = 2 0 0
31      Spin = 1
32      Ras2 = 4
33      ciroot = 2 2 1
34      RLXRoot = 1
35 EOF
36
37 # OpenMolcas launcher
38 ${OPENMOLCAS_HOME}/pymolcas -np $MOLCAS_NPROCS h2.inp > h2_${dist}.log 2>&1
39
40 # extract the two state energies from the log
41 #mapfile -t energies < <(awk '/RASSCF root number/ {root=$NF} /Total energy/ {
42   ↪ print root, $NF}' h2_${dist}.log | awk '$1==1 || $1==2 {print $2}')
43 mapfile -t energies < <(
44   awk '/RASSCF root number/ {print $NF}' h2_${dist}.log
45 )
46
47 es0=${energies[0]}
48 es1=${energies[1]}
49 gap=$(awk -v e0="$es0" -v e1="$es1" 'BEGIN{print (e1-e0)/2}')
50
51 printf "%s %s %s %s\n" "$dist" "$es0" "$es1" "$gap" | tee -a gaps.dat
done

```

The output surfaces are plotted in Figure 1.7, the halved energy gap in Figure 1.8. The minimum gap of $0.162 E_h$ is found at an interatomic distance of $2.20 \text{ \AA}$, even if the two surfaces do not show a proper “avoided crossing” behavior. *This mean that probably I will not see any population dynamics in TENSO. No crying on spoiled milk.*

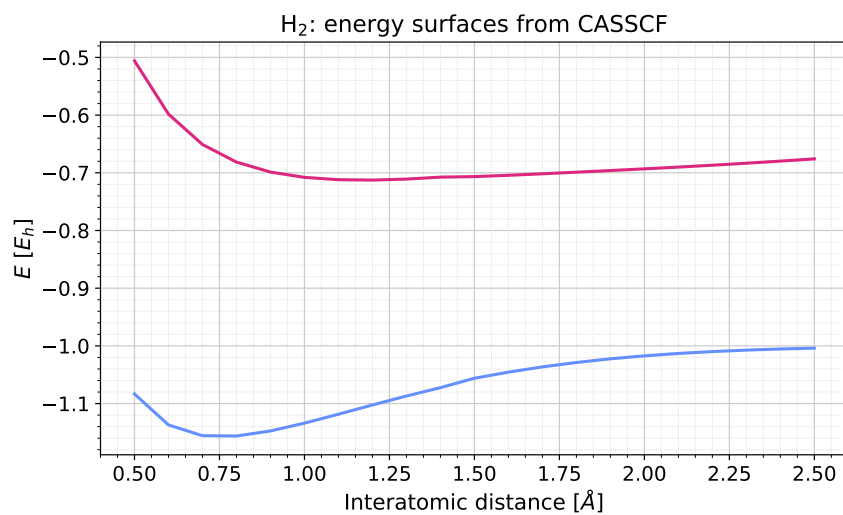


FIGURE 1.7: S_0 and S_1 potential energy surfaces of H_2 , obtained by surface scan with CASSCF on B3LYP def2-TZVP level of theory

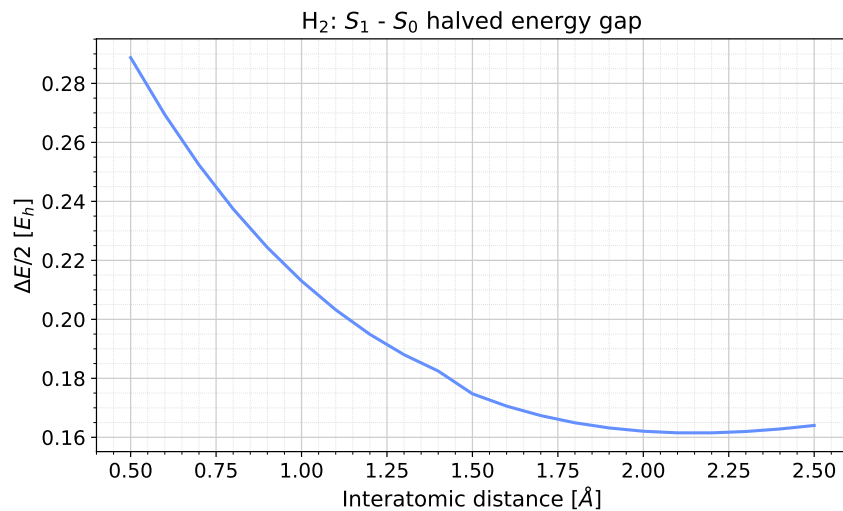


FIGURE 1.8: Halved energy gap of H_2 , obtained by surface scan with CASSCF on B3LYP def2-TZVP level of theory

1.6 THE TOY HAMILTONIAN

I want to model the H_2 system as two coupled harmonic oscillators surrounded by the C_{60} cage. Since I don't expect a lot of electronic transitions, I will have the $\nu = 0$ and $\nu = 1$ harmonic states to both S_0 and S_1 . This means that the system is represented by four states: $|S_0, \nu_0\rangle$, $|S_0, \nu_1\rangle$, $|S_1, \nu_0\rangle$ and $|S_1, \nu_1\rangle$. This makes the Hamiltonian a little more complicated than I planned, especially for the off-diagonal terms. The two harmonic oscillators are separated by a gap ΔE taken at the minimum of S_0 . I will take the $\min(\Delta E/2)$ coming from the OPENMOLCAS scan as diabatic coupling W between $|S_0, \nu_0\rangle$ and $|S_1, \nu_0\rangle$, leaving all the vibrational states uncoupled between each other. This is consistent with the harmonic picture, and leaves most of the transitions (if any) in the hands of the bath. The final system Hamiltonian will look like this:

$$\mathbb{H} = \begin{bmatrix} \frac{\omega_0}{2} & 0 & W & 0 \\ 0 & \frac{3\omega_0}{2} & 0 & 0 \\ W & 0 & \Delta E + \frac{\omega_1}{2} & 0 \\ 0 & 0 & 0 & \Delta E + \frac{3\omega_1}{2} \end{bmatrix}$$

The OPENMOLCAS outputs give me the values reported in Table 1.1. Using these values, I obtain the following toy Hamiltonian (in cm^{-1}):

$$\mathbb{H} = \begin{bmatrix} 2138 & 0 & 35447 & 0 \\ 0 & 6414 & 0 & 0 \\ 35447 & 0 & 108248 & 0 \\ 0 & 0 & 0 & 109973 \end{bmatrix}$$

Unfortunately, my analysis does not provide any sound way to build the system-bath Hamiltonian, so I will use this *ansatz*:

$$\mathbb{H}_{sb} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -0.5 \end{bmatrix}$$

in which the bath couples differently with S_0 and S_1 , and their respective vibrational states.

TABLE 1.1: *Summary of the OPENMOLCAS outputs employed to build the system toy Hamiltonian (in cm^{-1})*

ΔE	W	ω_1	ω_2
107366	35447	4276	1725

1.7 THE TOY SPECTRAL DENSITY

Since I could not obtain an actual spectral density, I recreate one that is compatible with C_{60} VDOS. Fortunately, C_{60} VDOS can be approximated by three major features, a broad one at 600 cm^{-1} ,

and two narrow ones at 1250 cm^{-1} and 1480 cm^{-1} . To fine-tune the model, I use small PYTHON script that just computes the two components of the spectral density $J(\omega)$:

$$J_{dl}(\omega) = \frac{2\lambda}{\pi} \frac{\omega_c \omega}{\omega^2 + \omega_c^2} \quad J_{bo}(\omega) = \frac{\lambda}{\pi} \frac{\eta \omega_0^2 \omega}{(\omega^2 - \omega_0^2)^2 + 4\eta^2 \omega^2}$$

given λ_{dl} , ω_c , ω_0 , λ_{bo} and η . Since I have no way to determine either λ , I will stick to TENSOR **input generator** values of $\lambda_{dl} = 540\text{ cm}^{-1}$ and $\lambda_{bo} = 330\text{ cm}^{-1}$. The broad feature is described by a Brownian oscillator at $\omega_0 = 650\text{ cm}^{-1}$ with a width of $\eta = 100\text{ cm}^{-1}$, while the narrow features are given by a Brownian oscillator at $\omega_0 = 1250\text{ cm}^{-1}$ with a width of $\eta = 50.0\text{ cm}^{-1}$ and another at $\omega_0 = 1480\text{ cm}^{-1}$ with a width of $\eta = 50.0\text{ cm}^{-1}$. To represent the baseline of the VDOS, I used a Drude-Lorentz component with $\omega_c = 700\text{ cm}^{-1}$, even if a Drude-Lorentz component with $\omega_c = 70.0\text{ cm}^{-1}$ (to include the roto-translations and librations) or no Drude-Lorentz component at all would probably be more fitting to the nature of the system. The three Brownian components are compared with the C_{60} VDOS in Figure 1.9. The Drude-Lorentz component is compared with the VDOS in Figure 1.10. The spectral density parameters are summarized in Table 1.2.

TABLE 1.2: *Summary of the spectral density parameters (in cm^{-1})*

Comp.	λ	ω_c / ω_0	η
DL	540	700	—
BO 1	330	650	100
BO 2	330	1250	50
BO 3	330	1480	50

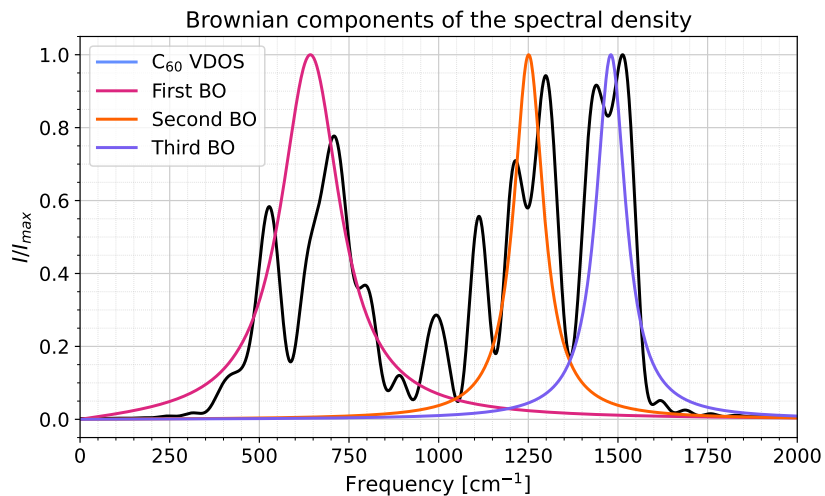


FIGURE 1.9: *Comparison between the three chosen Brownian oscillator components to the spectral density and the C_{60} VDOS of the $H_2 \cdot C_{60}$ system*

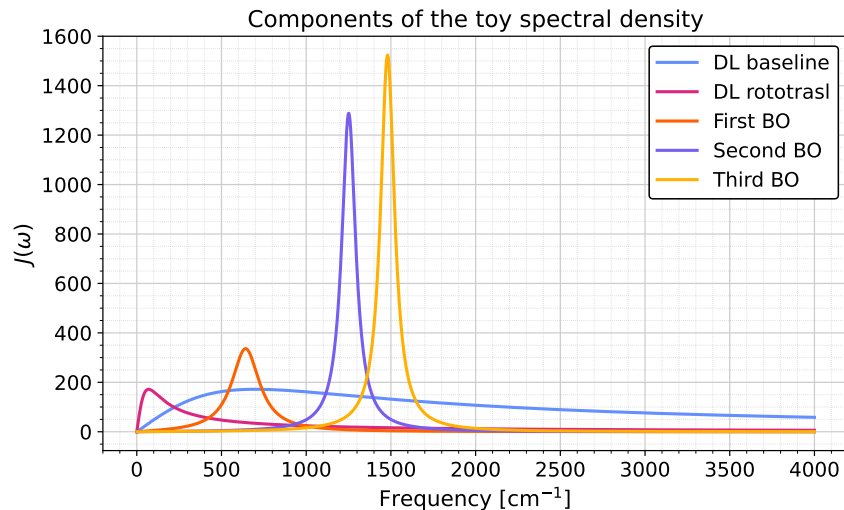


FIGURE 1.10: *Components of the toy spectral density*

1.8 EXPLORATORY TENSO SIMULATION

I generated the TENSO input with the provided [input generator](#), inserting my values. For the initial state, I decided to use the following density matrix:

$$\rho_0 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.5 \end{bmatrix}$$

that places my H_2 in quite a peculiar state, and I hope that something interesting comes from it! As initial temperature, I will keep 300 K. I will evolve only 100 fs with $dt = 0.100$ fs. Since the website flags a memory-heavy simulation, I first check if something actually happens in my system by evolving a short high-level simulation. This simulation confirms my fears that with my *ansatz* I should not expect a lot of dynamics. So I set the number of dimensions to 5 and the `ps_method` to `ps=1`, to at least save time. The following listing summarizes the setting I used.

```

1 # Bath
2 bath = gen_bcf(
3     re_d   = [540],
4     width_d = [700],
5     freq_b  = [650, 1252, 1480],
6     re_b    = [330, 330, 330],
7     width_b = [100, 50, 50],
8     temperature = 300,
9     decomposition_method = 'Pade',
10    n_ltc      = 1,
11 )
12
13
```



```

14 # System
15 sys_ham = np.array([
16     [2138, 0, 35477, 0],
17     [0, 6414, 0, 0],
18     [35447, 0, 108248, 0],
19     [0, 0, 0, 109973],
20 ], dtype=np.complex128)
21
22 sys_op0 = np.array([
23     [1, 0, 0, 0],
24     [0, 0.5, 0, 0],
25     [0, 0, -1, 0],
26     [0, 0, 0, -0.5],
27 ], dtype=np.complex128)
28
29 sys_ops = [sys_op0]
30
31 init_rdo = np.array([
32     [0, 0, 0, 0],
33     [0, 0.5, 0, 0],
34     [0, 0, 0, 0],
35     [0, 0, 0, 0.5],
36 ], dtype=np.complex128)
37
38
39 # Run
40 end_time = 100 # fs
41 dt = 0.1 # fs
42 out = 'output' # outputs: {out}.dat.log (data) and {out}.debug.log
43
44 propagator = system_multibath(
45     fname=out,
46     init_rdo=init_rdo,
47     sys_ham=sys_ham,
48     sys_ops=sys_ops,
49     bath_correlations=[bath],
50     end_time=end_time,
51     step_time=dt,
52     dim=5,
53     max_auxiliary_rank=32,
54     ode_atol=1e-9,
55     ode_rtol=1e-5,
56     frame_method='train',
57     rank=1,
58     stepwise_method='simple',
59     ps_method='ps1',
60 )

```

The results I obtained are not very exciting, confirming the chemical intuition and the early results: under these conditions, no population transfer is happening (Figure 1.11).

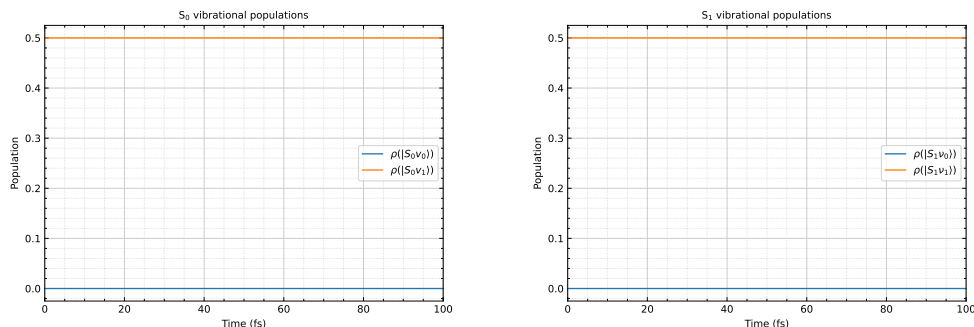


FIGURE 1.11: Population “dynamics” from TENS0 output; left: the two vibrational states of S_0 ; right: the two vibrational states of S_1 ; obtained by modifying the plot scripts provided in the tutorials

1.9 CONCLUSIONS

This project confirms the old saying that “an hour in the library saves a week in the lab”. Indeed, the aim of this project is to study if a workflow developed for computing VDOS can be extended to construct effective spectral density. However, the model system I tried to create is not at all a good benchmark. First of all, H_2 and C_{60} do not seem coupled, even if the H_2 stretching is heavily red-shifted. This prevented me to properly investigate if the QM energy fluctuations are a good way to analyze the system-bath coupling. Second, H_2 is probably the worst system to study vibronic transitions, given the energies at play. This forced me in the end to use an *ansatz* Hamiltonian part informed, part guessed, and all boring.

As for the workflow, `runner.py` is a good starting point to improve some of the issues I had with ORCA and OPENMOLCAS. First of all, from ORCA outputs I was able to recover only the total ONIOM energy or the QM1. This again stresses the importance of having greater control over the *ab initio* outputs. As for OPENMOLCAS, a PYSCF script would have probably handled the H_2 surface scan much better. This time I was lucky that I was working with H_2 . As anticipated, I will continue working on `runner.py`. First, I would like to include an Adiabatic Switching version, that requires to slowly changing the Hamiltonian from fully harmonic to fully *ab initio*. Then I would love to use PYSCF flexibility to use other *ab initio* packages to compute the forces. Finally, I think it would be interesting to find out what a quasiclassical VDOS of an excited state could tell us.

As for the TENS0 results, they are fully consistent with the model Hamiltonian. Nonetheless, the aim of this project was obtaining some results, not a breakthrough, and I am satisfied with what I got. Computing the spectral density and, more in general, the coupling picture from an *ab initio* trajectory remains an open problem, worthy of investigation.

1.10 CONTENTS OF THE FOLDER

- This file, both in `.pdf` and in `.md`; this file is the usual “lab report” that I keep for myself, just more curated than usual; I hope it was an enjoyable read
- In `/runner/`, I include the input (`geometry.xyz` and `velocity.xyz`) and output (`traj.xyz`, `forces.dat`)

and `md.log` files for H_2 (`/h2/`) and H_2O (`/h2o/`); in `/h2o/`, I also include the input necessary to compute the Heller Frozen Gaussian VDOS of Figure 1.2.

- In `/orca/`, I include the initial guess geometry, and input and relevant output files for ORCA optimization and frequencies job (`opt-freq`) and *ab initio* molecular dynamics (`dyn`); the files are divided in three folder based on the system: C_{60} (`/C60_XTB/`), $\text{H}_2 \cdot \text{C}_{60}$ (`H2atC60_ONIOM`) and H_2 (`H2_ORCA`).
- In `/dragonball/`, I include the final output files of the Dragonball runs, for C_{60} (`/c60_xtb/`), $\text{H}_2 \cdot \text{C}_{60}$ (`h2atc60`) and H_2 (`h2_orca`).
- In `/openmolcas/`, I include the input and relevant output files for the S_0 optimization and frequency run (`H2_S0_optfreq`), the S_1 optimization and frequency run (`H2_S1_optfreq`), and the surface scan (`H2_avoided`).
- In `/plot_specden/`, I include the script I used to compute the spectral densities (`sd.py`), its output files (`*.dat`) and the final plots (`*.pdf`).
- In `/tenso/`, I include the final run input (`final_run.py`) and output (`output.dat.log`), together with the modified plotting script (`plot_tenso.py`) and the two output plots (`s0.pdf` and `s1.pdf`).